

## Timer driver

The timer driver is responsible for managing timeouts. That is, user-land code needs to be able to define a time interval and a callback function, and have that callback function executed after the given interval has elapsed.

We achieve this by registering a timeout on the timer device in the OdroidC2's S905 SOC. This is done by defining a counter  $n$ , a timebase  $b$ , and a callback function. This will result in the callback function being triggered after  $n$  iterations of  $b$  has elapsed. For instance, if we set  $n = 100$  and  $b = \text{ms}$ , then the callback function will be triggered after 100 milliseconds.

## Challenge

The primary challenge for this therefore becomes the limited register size for the counter. For instance, Timer A has a counter size of 16 bits, which means that the maximum counter value is 65535. Consequently, selecting a timebase becomes a tradeoff decision between higher precision and higher maximum duration (and therefore less interrupts required). We take the approach of selecting the finest timebase we can that can fit the target duration.

## Design

We implement the timer driver with a tickless design. We maintain the active timers in a priority queue, such that the head is always the earliest-due timer. When registering a timer, we peek at the head, and see if it is due. If it is, then we pop it off of the queue, and trigger its callback. Otherwise, we re-arm the timer device with the finest timebase possible, and calculate the counter value accordingly.